

**INSTITUTO
FEDERAL**

Santa Catarina

Câmpus
São José

Relatório A3.1

Proposta 4: Simulador M/M/k/k com DES e Erlang-B

Curso: Engenharia de Telecomunicações

Disciplina: Avaliação de Desempenho de Sistemas

Professor: Eraldo Silveira e Silva

Aluno(s)

Wagner Flores dos Santos

Gabriel Kuster Kraus

16 de Maio de 2026

Sumário

1	Introdução	2
2	Objetivo	2
3	Características do sistema	2
4	Tecnologias Utilizadas	2
5	Estrutura do Projeto	3
6	Implementação da Simulação	3
6.1	Estrutura Geral do Simulador	3
6.2	Classe Event	4
6.3	Classe ArrivalEvent	4
6.4	Classe DepartureEvent	4
6.5	Classe Simulator	4
6.6	Geração de Números Pseudoaleatórios	5
6.7	Interface Gráfica	5
6.8	Métricas Coletadas	6
7	Resultados Obtidos	6
7.1	Experimentos do Enunciado	6
7.2	Reprodutibilidade dos Resultados	7
7.3	Exemplo de Teste com Baixa Carga	8
7.4	Exemplo de Teste com Alta Carga	9
7.5	Análise dos Resultados	9
8	Repositório do Projeto	10

1 Introdução

O sistema implementa uma Simulação a Eventos Discretos (DES) de um sistema celular do tipo M/M/k/k, comparando os resultados simulados com os resultados teóricos obtidos pela fórmula de Erlang-B.

2 Objetivo

O projeto simula:

- chegadas de chamadas segundo processo de Poisson;
- duração das chamadas com distribuição exponencial;
- múltiplos canais de comunicação;
- bloqueio imediato de chamadas quando todos os canais estão ocupados;
- cálculo de métricas de desempenho;
- comparação entre teoria e simulação.

3 Características do sistema

Tabela 1: Características do modelo M/M/k/k

Característica	Descrição
M	Chegadas Poisson
M	Serviço exponencial
k	Número de canais
k	Capacidade máxima do sistema
Fila	Não possui fila de espera
Bloqueio	Imediato quando todos os canais estão ocupados

4 Tecnologias Utilizadas

O simulador foi desenvolvido utilizando a linguagem Python e bibliotecas voltadas para computação científica, visualização de dados e construção de interfaces web.

As principais tecnologias utilizadas foram:

- Python 3;
- Streamlit;
- Plotly;
- Pandas;
- NumPy.

A linguagem Python foi utilizada para implementação da lógica da simulação a eventos discretos e dos cálculos estatísticos.

A biblioteca Streamlit foi utilizada para construção da interface gráfica interativa, permitindo a configuração dos parâmetros da simulação e a visualização dos resultados em tempo real.

A biblioteca Plotly foi utilizada para geração dos gráficos comparativos entre os resultados teóricos e simulados.

Já as bibliotecas Pandas e NumPy foram empregadas no armazenamento, manipulação e processamento dos dados gerados durante os experimentos.

5 Estrutura do Projeto

O projeto foi organizado em módulos independentes, permitindo separar a lógica da simulação, os cálculos teóricos e a interface gráfica.

A estrutura de diretórios utilizada é apresentada a seguir:

Tabela 2: Arquivos do projeto

Arquivo	Função
app.py	Interface gráfica Streamlit
simulator.py	Núcleo da simulação DES
events.py	Eventos da simulação
erlang.py	Cálculo de Erlang-B
experiment.py	Execução dos experimentos
plot_results.py	Geração de gráficos
requirements.txt	Dependências do projeto
imagens/	Figuras utilizadas
results/	Resultados exportados
README.md	Documentação do projeto

Essa organização modular facilita a manutenção do código, a reutilização dos componentes e a validação individual das funcionalidades implementadas.

6 Implementação da Simulação

A simulação foi desenvolvida em Python utilizando a técnica de Simulação a Eventos Discretos (Discrete Event Simulation - DES), permitindo reproduzir o comportamento de um sistema celular modelado como uma fila do tipo (M/M/9/9).

O sistema foi organizado em módulos independentes, separando a lógica dos eventos, do simulador principal, dos cálculos teóricos e da interface gráfica. Essa organização facilitou a implementação, manutenção e validação dos resultados obtidos.

6.1 Estrutura Geral do Simulador

A simulação é baseada em uma fila de eventos futuros, na qual cada evento possui um instante de ocorrência associado.

Os principais eventos considerados foram:

- chegada de uma chamada (`ArrivalEvent`);

- término de uma chamada (`DepartureEvent`).

Os eventos são armazenados em uma fila de prioridade implementada com a biblioteca `heapq`, permitindo que sejam processados em ordem cronológica.

6.2 Classe Event

A classe `Event` representa um evento genérico da simulação.

Cada evento possui um atributo `time`, que indica o instante em que deverá ocorrer, e um método `process()`, responsável por executar as ações associadas ao evento.

Além disso, foi implementado o método `__lt__()`, permitindo que os eventos sejam ordenados automaticamente pela fila de prioridade.

6.3 Classe ArrivalEvent

A classe `ArrivalEvent` representa a chegada de uma nova chamada ao sistema.

Quando uma chegada ocorre, o simulador executa as seguintes etapas:

1. contabiliza uma nova chamada gerada;
2. verifica se existe canal disponível;
3. aceita ou bloqueia a chamada;
4. gera a duração da chamada;
5. agenda o evento de término da chamada;
6. agenda a próxima chegada.

Caso exista pelo menos um canal livre, a chamada é aceita e passa a ocupar um dos canais disponíveis.

Quando todos os canais estão ocupados, a chamada é imediatamente bloqueada, característica típica dos sistemas Erlang-B.

6.4 Classe DepartureEvent

A classe `DepartureEvent` representa o término de uma chamada.

Quando o evento é processado, o canal anteriormente ocupado é liberado, tornando-se novamente disponível para atender futuras chamadas.

Essa operação é realizada através da redução do número de canais ocupados pelo sistema.

6.5 Classe Simulator

A classe `Simulator` é responsável pelo controle completo da execução da simulação.

Suas principais funções são:

- manter a fila de eventos futuros;
- processar eventos em ordem cronológica;
- controlar os canais ocupados;
- coletar métricas de desempenho;
- calcular utilização média;

- calcular vazão do sistema;
- calcular probabilidade de bloqueio.

A execução da simulação inicia com o agendamento da primeira chegada de chamada no instante ($t=0$).

A partir desse momento, os eventos passam a ser processados sucessivamente até que o tempo máximo de simulação seja atingido.

6.6 Geração de Números Pseudoaleatórios

A geração dos números pseudoaleatórios foi realizada utilizando a biblioteca padrão `random` do Python.

Internamente, essa biblioteca utiliza o algoritmo Mersenne Twister (MT19937), amplamente empregado em aplicações de simulação devido à sua elevada qualidade estatística e longo período de repetição.

Os tempos entre chegadas e os tempos de permanência das chamadas foram gerados através da função:

```
random.expovariate()
```

Essa função produz variáveis aleatórias com distribuição exponencial a partir dos números pseudoaleatórios uniformes gerados pelo PRNG.

Foram utilizadas duas distribuições exponenciais distintas:

- intervalo entre chegadas das chamadas, parametrizado por (λ);
- duração das chamadas, parametrizada por (μ).

Para garantir a reprodutibilidade dos experimentos, foi implementado o uso de uma semente (*seed*) configurável pelo usuário.

Dessa forma, execuções realizadas com os mesmos parâmetros e a mesma semente produzem exatamente os mesmos resultados.

6.7 Interface Gráfica

Foi desenvolvida uma interface gráfica utilizando a biblioteca Streamlit.

A interface permite ao usuário alterar dinamicamente os parâmetros do sistema:

- taxa média de chegada (λ);
- taxa média de serviço (μ);
- número de canais (k);
- tempo de simulação;
- semente do gerador pseudoaleatório.

Além da execução de simulações interativas, a aplicação também possui uma seção destinada à execução automática dos cenários especificados no enunciado, permitindo comparar os resultados simulados com aqueles obtidos pela fórmula de Erlang-B.

6.8 Métricas Coletadas

Durante a execução da simulação são coletadas as seguintes métricas:

- número total de chamadas geradas;
- número total de chamadas aceitas;
- número total de chamadas bloqueadas;
- probabilidade de bloqueio simulada;
- utilização média dos canais;
- vazão do sistema.

A probabilidade de bloqueio simulada é calculada por:

$$P_b = \frac{\text{Chamadas Bloqueadas}}{\text{Chamadas Geradas}}$$

A utilização média dos canais é obtida através da integração da ocupação dos canais ao longo do tempo de simulação.

Já a vazão é calculada como a razão entre o número de chamadas aceitas e o tempo total de execução da simulação.

7 Resultados Obtidos

7.1 Experimentos do Enunciado

A aplicação desenvolvida executa automaticamente os cenários especificados no enunciado do trabalho, permitindo comparar os resultados obtidos pela Simulação a Eventos Discretos (DES) com os valores teóricos calculados pela fórmula de Erlang-B.

Os experimentos foram realizados considerando:

- $\lambda = 2, 4, 5, 6, 8$;
- $\mu = 1$;
- $k = 9$ canais.

Para cada cenário foram calculadas as seguintes métricas:

- probabilidade de bloqueio teórica (Erlang-B);
- probabilidade de bloqueio simulada (DES);
- erro absoluto;
- utilização média dos canais;
- vazão do sistema.

Os experimentos utilizam uma semente configurável para o gerador pseudoaleatório, permitindo total reprodutibilidade dos resultados obtidos.

A Figura 1 apresenta a tabela de resultados gerada automaticamente pela aplicação.



Figura 1: Tabela de resultados dos experimentos automáticos.

A Figura 2 apresenta a comparação entre os resultados teóricos obtidos pela fórmula de Erlang-B e os resultados simulados pela técnica DES.

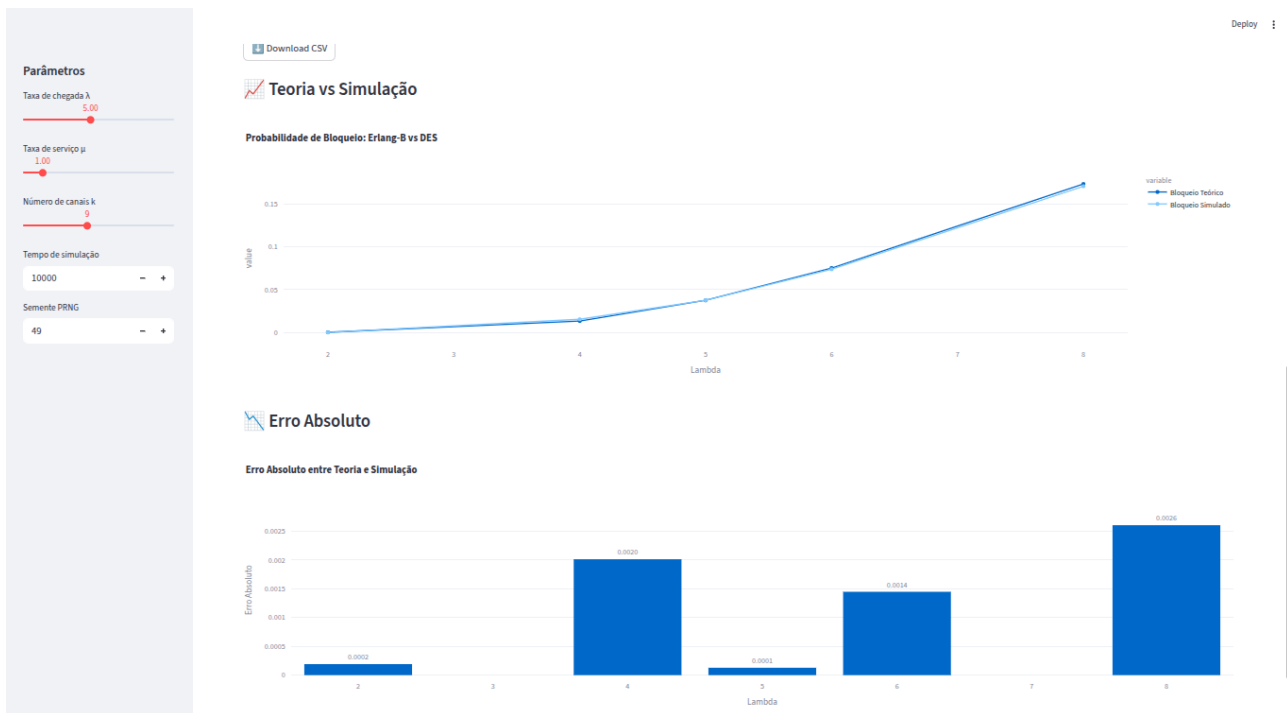


Figura 2: Comparação entre resultados teóricos e simulados.

Observa-se que os valores simulados acompanham adequadamente a tendência prevista pelo modelo teórico, demonstrando a correta implementação do simulador.

7.2 Reprodutibilidade dos Resultados

A simulação utiliza um gerador pseudoaleatório (PRNG) inicializado por uma semente configurável pelo usuário.

Dessa forma:

- mesma semente e mesmos parâmetros produzem exatamente os mesmos resultados;

- sementes diferentes produzem sequências pseudoaleatórias distintas e, consequentemente, resultados diferentes.

Essa característica permite reproduzir integralmente os experimentos realizados, sendo uma prática importante em estudos de simulação computacional.

Embora os eventos sejam gerados de forma aleatória, a utilização de uma semente fixa garante a repetibilidade das execuções.

7.3 Exemplo de Teste com Baixa Carga

O primeiro experimento foi realizado utilizando uma carga relativamente baixa para o sistema.

Tabela 3: Parâmetros do teste com baixa carga.

Parâmetro	Valor
λ	2
μ	1
k	9
Tempo de Simulação	10000
Semente	42

Nesse cenário espera-se uma baixa probabilidade de bloqueio, uma vez que a taxa de chegada de chamadas é significativamente inferior à capacidade total do sistema.



Figura 3: Resultado obtido para o cenário de baixa carga.

Observa-se que o bloqueio é praticamente inexistente e que a utilização média dos canais permanece relativamente baixa.

7.4 Exemplo de Teste com Alta Carga

O segundo experimento foi realizado utilizando uma carga mais elevada.

Tabela 4: Parâmetros do teste com alta carga.

Parâmetro	Valor
λ	8
μ	1
k	9
Tempo de Simulação	10000
Semente	42

Nesse caso espera-se um aumento significativo da probabilidade de bloqueio, uma vez que a taxa de chegada de chamadas aproxima-se da capacidade máxima do sistema.

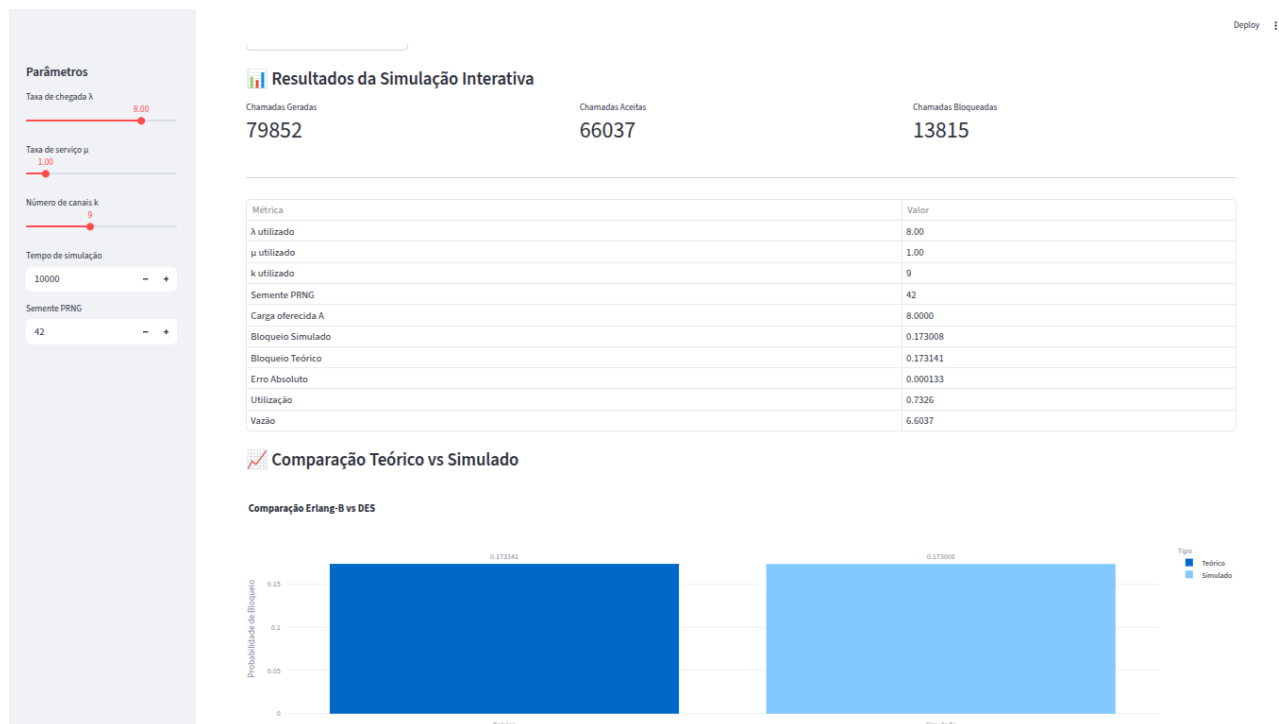


Figura 4: Resultado obtido para o cenário de alta carga.

Observa-se um aumento considerável da utilização dos canais e da probabilidade de bloqueio, comportamento compatível com o previsto pela teoria de Erlang-B.

7.5 Análise dos Resultados

Os resultados obtidos demonstram que a probabilidade de bloqueio cresce à medida que a carga oferecida ao sistema aumenta.

Além disso, verifica-se que os valores simulados permanecem próximos dos valores teóricos em todos os cenários analisados, indicando que a implementação da Simulação a Eventos Discretos reproduz adequadamente o comportamento esperado para sistemas do tipo (M/M/k/k).

Os erros absolutos observados permaneceram reduzidos, demonstrando boa convergência entre teoria e simulação.

Por fim, observou-se que o aumento do tempo de simulação tende a reduzir as diferenças entre os resultados simulados e teóricos, comportamento esperado para sistemas estocásticos modelados por Simulação a Eventos Discretos.

8 Repositório do Projeto

O código-fonte completo desenvolvido para este trabalho está disponível no GitHub no seguinte endereço:

<https://github.com/wagnerfloresdossantos/engtelecom-ifsc-sj/tree/main/ads-2026.1/a3.1>

O repositório contém todos os arquivos utilizados no desenvolvimento do simulador, incluindo:

- Código-fonte da simulação a eventos discretos;
- Implementação da fórmula de Erlang-B;
- Interface gráfica desenvolvida em Streamlit;
- Scripts para execução dos experimentos;
- Arquivos de teste;
- Documentação do projeto (README.md).

O projeto foi versionado utilizando Git, permitindo rastreabilidade das alterações e reprodutibilidade dos experimentos realizados.

Referências

MARZOLLA, Moreno. *The Queueing Network Analysis Package for Octave*. Disponível em: https://www.moreno.marzolla.name/software/queueing/queueing.html#The-M_002fM_002f1-System.

Acesso em: 16 maio 2026.

SILVA, Eraldo Silveira e. *Conceitos Básicos em Simulação a Eventos Discretos*. Disciplina Avaliação de Desempenho de Sistemas (ADS29009). Instituto Federal de Santa Catarina (IFSC), São José, 2026.

SILVA, Eraldo Silveira e. *Geração de Números Randômicos na Simulação*. Disciplina Avaliação de Desempenho de Sistemas (ADS29009). Instituto Federal de Santa Catarina (IFSC), São José, 2026.

SILVA, Eraldo Silveira e. *Rede de Filas*. Disciplina Avaliação de Desempenho de Sistemas (ADS29009). Instituto Federal de Santa Catarina (IFSC), São José, 2026.

SANTOS, Wagner; KRAUS, Gabriel. *Simulador M/M/k/k com Erlang-B e Simulação a Eventos Discretos*. GitHub, 2026. Disponível em: <https://github.com/wagnerfloresdossantos/engtelecom-ifsc-sj/tree/main/ads-2026.1/a3.1>. Acesso em: 30 maio 2026.